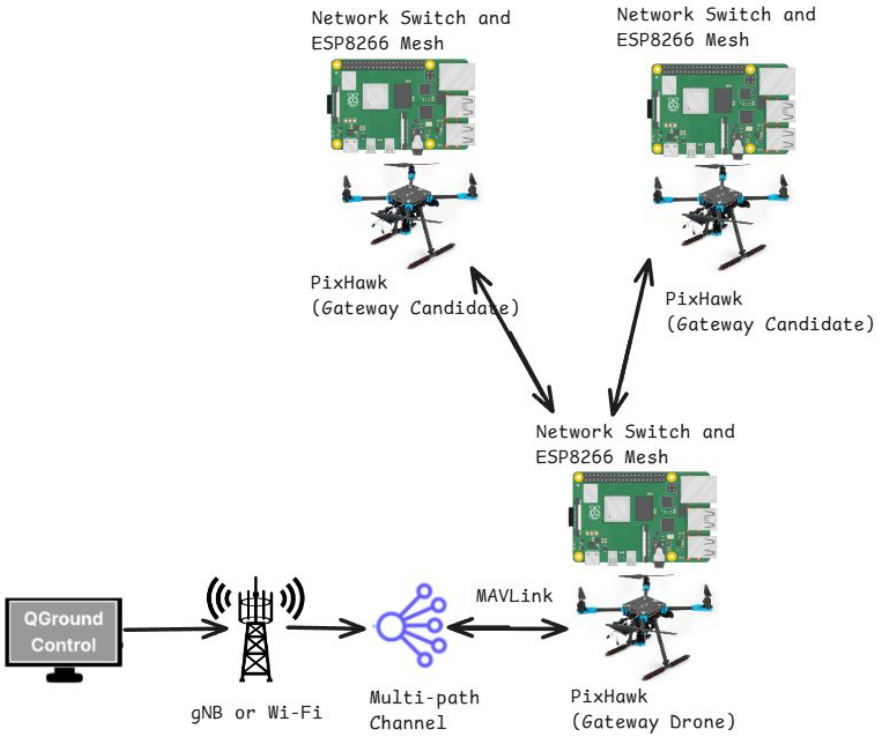
AeroShield Updates

10/31/25

PX4 - QGC - Pixhawk Connection Overview

This diagram shows how PX4 (or ArduPilot) based UAVs connect to QGroundControl through a multi path network. One drone will act as the gateway (providing connection and collecting logs from all UAVs on the network), linking the swarm to the ground control station via Wi-Fi or 5G (MAVLink). The other drones form a P2P mesh network, allowing them to share data and commands locally. If the gateway drone loses connection, one of the gateway candidates will automatically take over, ensuring that the swarm stays connected and the mission is maintained

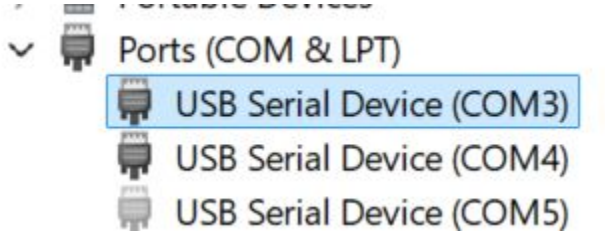


Connecting to Drones via usb

- If QGC doesn't work when just plugging the pixhawk through usb, some debugging steps below
- Had to install drivers since qgc wasn’t recognizing the plugged in pixhawk - pixhawk 6c apparently defaults to comm3 link, so when plugging it into your computer, go to device manager and then expand Ports(Com & LPT) (if not there go to view -> show hidden devices)
    - Then right click on com3 -> update drivers -> browse my computer for drivers -> let me pick from drivers on my computer -> have disk
        - From here for me (Dominic) I went program files and found that I had a folder for UAV drivers (i think this was installed by QGC? Not really sure where it came from) then i selected that and installed it
    - Then I plugged the pixhawk again and it auto connected. It said comms lost for a while (I think because I was messing around with it a lot to get it connected) so i just restarted qgc and it connected fine
- Also I just ran QGC from windows (instead of wsl) for simplicity, might just be better to do this in general anyways

Supporting Pictures

COM3 should not be greyed out, or else the usb is Not recognized



This is what it will look like when the drone is connected

More

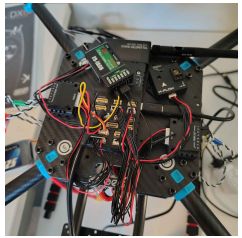
Arming/disarming works fine, but for some reason i'm not able to get any of the motors to spin

- If it gives arming errors go to settings -> vehicle config -> params -> type in safety then disable safety default (make sure to change this back for real flights!!!!)
- I disabled all safety checks (bc it was freaking out about calibration issues, and i tried to calibrate but i'm pretty sure there is magnetic interference in the lab since i kept getting an error saying that and aborting calibration)
- Took the drone off of safety mode by holding the switch button on the M10 (solid red - safety off, blinking red- safety on)

Also another note is that the drone seems to have ArduPilot not PX4, so maybe we switch if we really want to use PX4 we can flash the drone with that but i think we can just go to ArduPilot

Transmitter - Receiver Connection

- FS-I6X Transmitter, FS-IA6B 2.4 GHz 6 Channel Receiver
  - The transmitter can handle 10 channels but our receiver is 6 channels
  - Set up receiver
    - If the B/VCC cable (the looped cable thingy) is not set up vertically along the B/VCC channel part, position it so. Set up PPM cable similarly with the signal (yellow) wire is on top and the ground (black) wire is on the bottom.
    - Connect it to Pixhawk 6c PPM/SBUS RC port
  - Ensure the drone is powered
  - Power on transmitter (while drone is powered)
    - Make sure all the switches at the top of the device are switched upward and that the throttle (the left stick) is switched downward
    - Hold down on the bind button and toggle the power switch on
    - Keep holding until you see an RX On (or something similar that wasn't there before)
    - It will look like that third image
- Reference: https://www.youtube.com/watch?v=hHiYP7Sqypc



Connecting to the Drones Over WiFi

- Using HiLetgo ESP8266 NodeMCU Type-C
  - Wiring
    - **Might have to cut a few of the 6 wire UART cables to connect these to the drone?** (Need to check this before moving forward)
    - Well anyways, connect it to the drone in some fashion, serially connect to drone via usb to access this and flash it with MAVLink stuff (refer to links below)
      - Or perhaps flash it via a USB-C connection? Either way still needs to be connected to the drone – that method perhaps being via UART
    - Once flashed and set up and stuff should be able to connect to it via QGC using the credentials its set up with
  - Idea: So the plan is to get the esp hooked up to the drone, flash and configure it with qgc while serially connected, disconnect the serial connection, then connect to the drone via esp using qgc
- Reference:
  - https://docs.px4.io/main/en/telemetry/esp8266_wifi_module
  - https://www.nodemcu.com/index_en.html
  - https://www.youtube.com/watch?v=vcJedkna_aI

Working with the ESP8266

Install drivers
https://www.silabs.com/software-and-tools/usb-to-uart-bridge-vcp-drivers?tab=downloads

Then download the CP210x VCP Windows, unzip it and run the CP210xVCPInstaller_x64.exe

If you check in device manager -> Ports you should see Silicon Labs CP210x USB to UART Bridge, if it’s there your good

Now install esptool (pip install esptool) -> python -m esptool --port <COM PORT> chip-id

Flash -> python -m esptool --port <COM PORT> erase_flash

^^Don't really need to flash it unless there is something already running on it. If you want to upload new code you can do that and it should just overwrite the existing code

Working with the ESP8266 (simple test)

Now install Arduino IDE: https://www.arduino.cc/en/software/ (just makes it a lot easier)

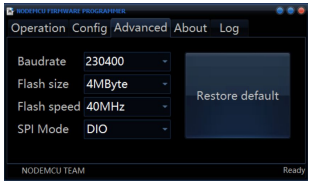
Once that's installed click on the top to select a board and type in Generic ESP8266 Module and your com port. It should then prompt you to install the packages for it and after that you should be connected. (Might have to go to File > Preferences > enter “http://arduino.esp8266.com/stable/package_esp8266com_index.json” into “Additional boards manager URLs”)

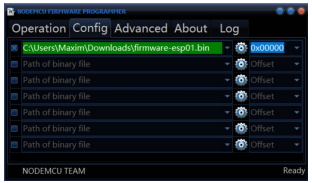
Tested the connection with this code (click verify then upload) and it should make the light blink on it ->

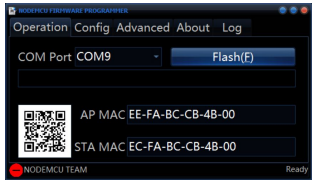
```
const int ledPin = LED_BUILTIN;  // Built-in LED, usually GPIO2
void setup() {
  pinMode(ledPin, OUTPUT);
}
void loop() {
  digitalWrite(ledPin, LOW);   // turn LED on
  delay(1000);
  digitalWrite(ledPin, HIGH);  // turn LED off
  delay(1000);
}
```

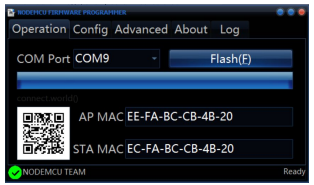
Flashing the ESP8266

- **Note: Mostly unnecessary, they come with firmware already but just for completeness' sake**
- Install NodeMCU Flasher:
  - https://github.com/nodemcu/nodemcu-flasher/blob/master/Win64/Release/ESP8266Flasher.exe
- Obtain firmware to be flashed:
  - https://github.com/BeyondRobotix/mavesp8266
    - If on mac just follow the commands
    - If on windows python -m pip install platformio then do the last 3 commands listed with the final one being python -m platformio run (or python3 however u did it)
  - The above failed to even compile for me so I just decided to run up one of these:
    - https://firmware.ardupilot.org/Tools/MAVESP8266/latest/
  - New best method (screw all that above):
    - Cloud based firmware builder (they'll email it to you after a min or two):
    - https://nodemcu-build.com/index.php
    - I just used the default settings
- Actually flashing it
  - Configure the Advanced and Config tabs as so
  - In Config, click the gear and select your firmware
  - In Operation, ensure you have the correct COM port and Flash
- Resources:
  - https://ardupilot.org/copter/docs/common-esp8266-telemetry.html
  - https://github.com/BeyondRobotix/mavesp8266
  - https://nodemcu.readthedocs.io/en/release/flash/









Uploading Code to the ESP

- **Note: Might have to do this multiple times as we develop the P2P**
- Get a NodeMCU uploader tool
  - Refer to the link below
  - I chose NodeMCU-Tool:
    - https://github.com/NodeMCU-Tool/NodeMCU-Tool
    - Can npm install it with: npm install nodemcu-tool -g
      - Ofc install npm beforehand etc
  - Arduino IDE also works nicely too (used for next slide)
- Resources:
  - https://nodemcu.readthedocs.io/en/release/upload/

Mesh Network

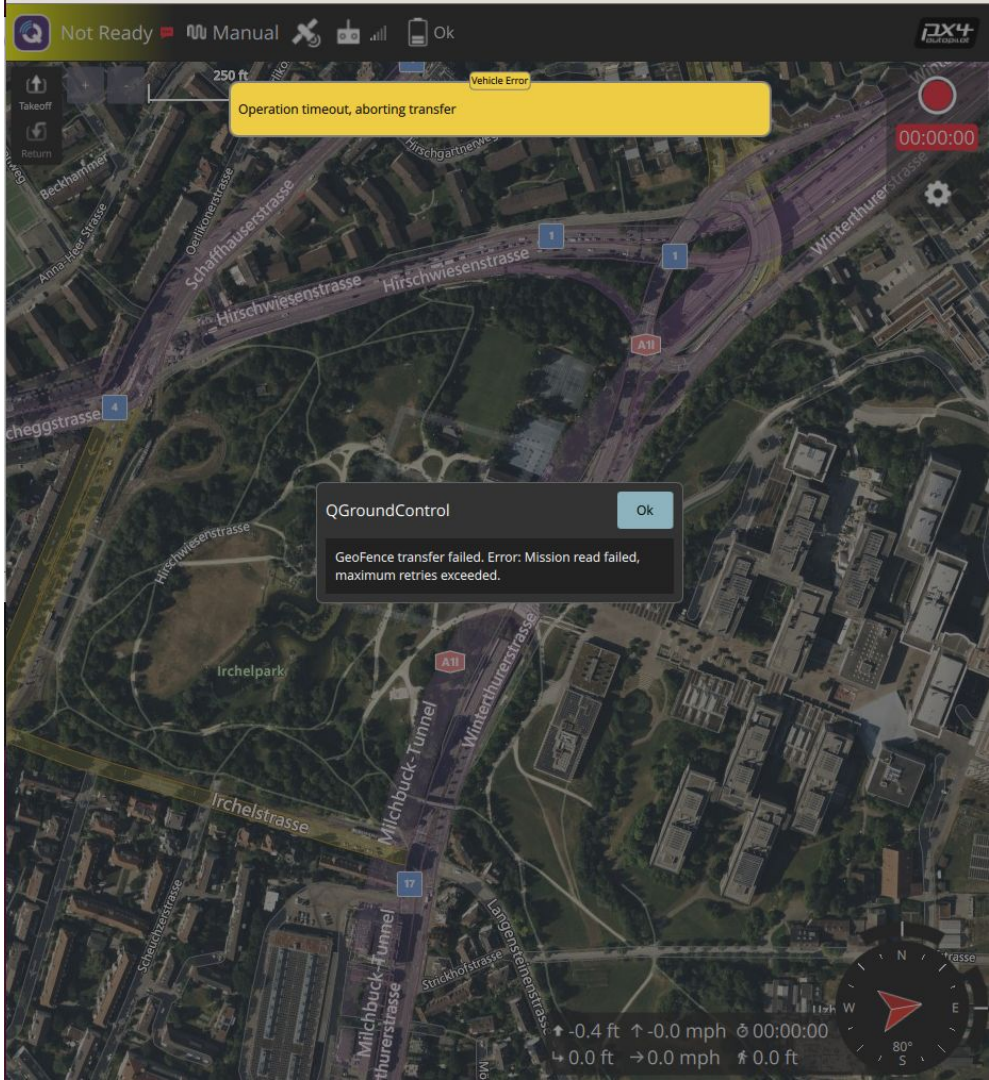
For this test, we set up three nodes (esp8266's) and uploaded the code (in the github). The mesh was successful and the nodes were able to send and receive messages

Reference: https://randomnerdtutorials.com/esp-mesh-esp32-esp8266-painlessmesh/?utm_source=perplexity

```
[node3] ~ time adjusted: offset=1956 now=56192382
[node3] <- from 3167439714 : hi from node1 id=3167439714 count=242 t=600450
[node3] <- from 3664036618 : hi from node2 id=3664036618 count=20 t=57232
[node3] <- from 3664036618 : hi from node2 id=3664036618 count=21 t=58697
[node3] <- from 3167439714 : hi from node1 id=3167439714 count=243 t=603756
[node3] <- from 3664036618 : hi from node2 id=3664036618 count=22 t=61214
[node3] <- from 3167439714 : hi from node1 id=3167439714 count=244 t=605408
```

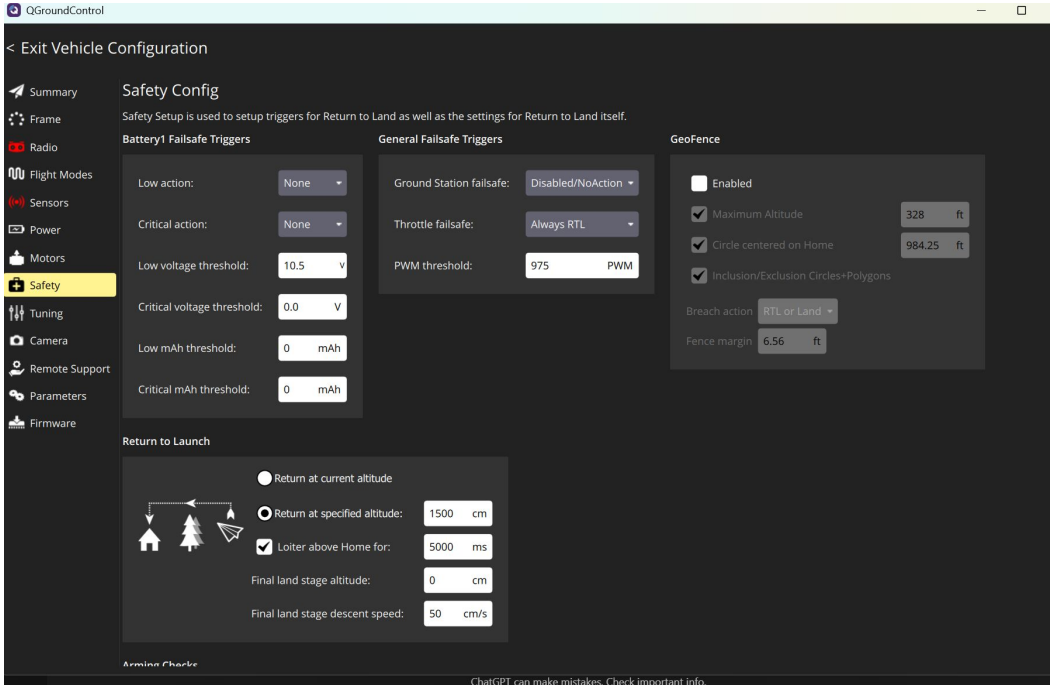
Node 3 is receiving the messages

To use, then follow previous slides (Working with the ESP8266 (simple test)) instructions to connect to and upload the code. Once code is on all nodes, make sure they are all powered and it should run automatically. To see output, in Arduino IDE go to tools -> Serial Monitor, then you should see the outputs from the nodes


Not Ready
Manual
Ok
Takeoff
Return
250 ft
Vehicle Error
Operation timeout, aborting transfer
00:00:00
Beckhammer
Anna-Heer-Strasse
Oerlikonerstrasse
Schaffhauserstrasse
Hirschwiesenstrasse
Hirschwiesenstrasse
Winterthurerstrasse
A11
4
QGroundControl
Ok
GeoFence transfer failed. Error: Mission read failed, maximum retries exceeded.
Irchelpark
Winterthurerstrasse
Milchbuck-Tunnel
Irchelstrasse
17
Scheuchzerstrasse
Milchbuck-Tunnel
Strickhofstrasse
Langensteinenstrasse
N
W
E
S
80°
-0.4 ft
-0.0 mph
00:00:00
0.0 ft
0.0 mph
0.0 ft

Default Configs to Remember (Testing)

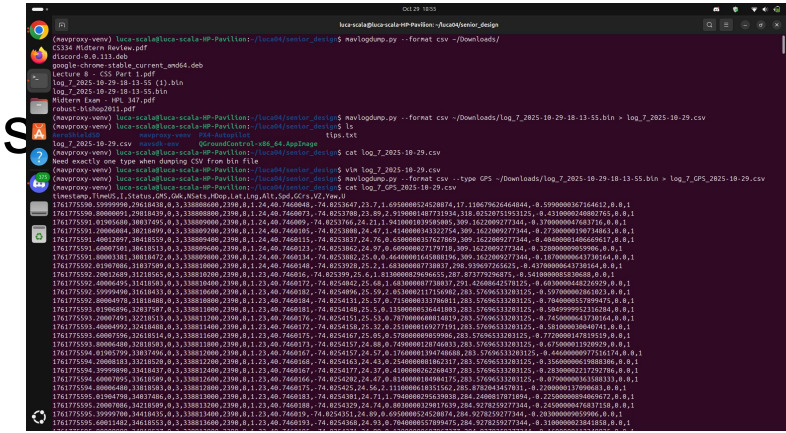
- Getting rid of safety for testing because I think the comms lost are coming from the fact that its telling me low voltage failsafe



Starting a Physical Mission and Extracting Data

- First step involved connecting controller to drone and following calibration steps
  - Moved left & right sticks into various positions to confirm connection
- Calibrating radio and gps sensors
  - Had to physically pick up drone and rotate in random directions to cover xyz
  - Rotated until calibration bar filled up
- Ran into “no gps lock” error that was eventually fixed by pressing vehicle gps button in qgc
- Ran into “Battery 1 low voltage failsafe” error
  - Despite trying different lipo batteries and also plugging power supply directly into wall, issue persists and prevented drone from being armed
- Fixed battery error by going and disabling battery 1 monitor
- “Hardware safety switch” error encountered
  - Fixed by disabling arming checks (should be ok since I’m just trying to run a mission and collect physical data)
- Was then able to arm and tried to take off but encountered a takeoff error
  - MAV_CMD_NAV_TAKEOFF command failed
- Then “GPS Glitching” error popped up
- To fix: simply restarted and re-calibrated accelerometer
- Ran into “Need position estimate” error
  - Fixed by recalibrating the compass successfully

Quick “Flight” and Downloaded Logs



- Was finally able to arm and “takeoff” with drone
  - No propellers so it stayed stationary obviously
- “Landed” successfully and downloaded log file from QGC
  - Generated log_7_2025-10-29-18-13-55.bin
- Took .bin file and used mavproxy mavlogdump.py to generate csv output based on what type of data I wanted
  - I specified GPS data but can specify these types as well PIDP, MCU, XKF2, RCI2, D32, CTRL, RATE, ATT, HEAT, AHR2, STAK, GPS, CTUN, RCOU, ARM, MAV, BARO, XKQ, UART, XKV2, PSCD, PIDE, DCM, MULT, MAG, UBX2, PSCN, IMU, ORGN, PM, XKF4, FILE, TERR, XKF3, ERR, FMT, PARM, POS, UNIT, VER, IOMC, RCO2, EAHR, POWR, XKF5, PIDN, RAD, GPA, EV, XKY1, FMTU, SURF, XKV1, MOTB, PIDA, XKY0, ANG, MSG, XKF1, XKFS, MODE, DU32, DSF, PIDR, MAVC, XKT, SRTL, RCIN, VIBE, LDET, MISE, PSCE, PIDY
- First: source ~/mavproxy-venv/bin/activate
- Then: mavlogdump.py --format csv --type TYPE_NAME your_log_file.bin > output.csv